

3-CSP LocalSearch

局部搜索

- 一开始就给所有变量赋值（即使违反约束）
- 在其解空间的**领域**内寻找更好的解（迭代改进Iterative Improvement）
- 需要一个评估函数对状态打分
- 直到满足终止条件

通用的框架

```
def local_search(initial_solution, max_iterations=1000):  
    """  
    局部搜索通用框架  
  
    Args:  
        initial_solution: 初始解  
        max_iterations: 最大迭代次数  
  
    Returns:  
        找到的最优解  
    """  
    current_solution = initial_solution  
    current_value = evaluate(current_solution)  
  
    for i in range(max_iterations):  
        # 1. 生成邻域  
        neighborhood = generate_neighbors(current_solution)  
  
        # 2. 评估邻域中的解  
        best_neighbor = None  
        best_neighbor_value = current_value  
  
        for neighbor in neighborhood:  
            neighbor_value = evaluate(neighbor)  
  
            # 寻找比当前解更好的邻域解  
            if neighbor_value > current_value: # 假设是最大化问题  
                best_neighbor = neighbor  
                best_neighbor_value = neighbor_value  
  
        # 3. 如果找到更好的解，则移动  
        if best_neighbor is not None:  
            current_solution = best_neighbor
```

```

        current_value = best_neighbor_value
    else:
        # 达到局部最优，停止
        break

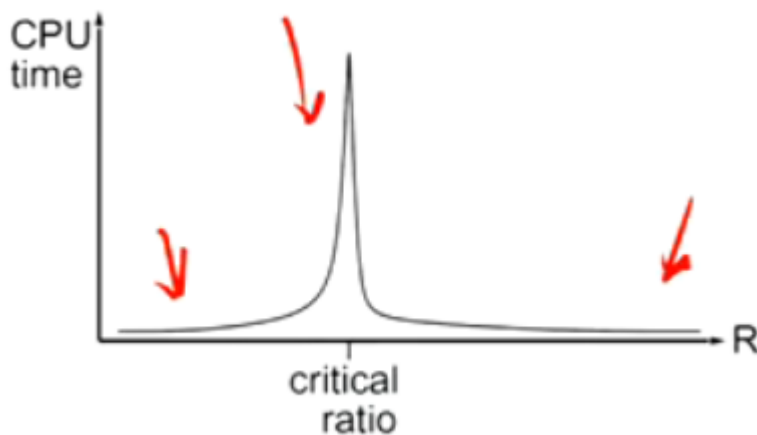
    return current_solution, current_value

```

在领域内寻找可能陷入**局部最优**，内存小速度快

对于N-Queen问题，如果使用类似的方法，随机初始状态，然后选择不满足的变量，移动到违背约束最少的地方，大约能解决 $n \sim 10\,000\,000$ 的N-Queen问题

根据 $R = \text{number of constraints} / \text{number of constraints}$ ，这种方法需要的时间会有很大不同



在约束很少的时候，一开始的随机赋值很可能就很接近答案；约束很多的时候，很容易调整那些不满足的变量达到最优（可以结合梯度下降法的函数图像理解）

爬山算法 Hill Climbing

```

def hill_climbing(initial_solution):
    """
    最陡上升爬山法 Steepest-Ascent Hill Climbing
    总是选择邻域中最好的解
    """
    current = initial_solution
    current_value = evaluate(current)

    while True:
        neighbors = get_neighbors(current)

        # 找到最好的邻居
        best_neighbor = None
        best_value = current_value

        for neighbor in neighbors:
            neighbor_value = evaluate(neighbor)
            if neighbor_value > best_value:
                best_neighbor = neighbor

```

```
best_value = neighbor_value
```

```
# 如果没有更好的邻居，停止
if best_value <= current_value:
    return current, current_value

# 移动到更好的解
current = best_neighbor
current_value = best_value
```

生成所有邻居之后，选择评估函数结果最好的一个，转移状态

但是邻居可能有很多个，为了降低找到最优的计算量，也有随机爬山法 Stochastic Hill Climbing，只要找到比当前状态更好的就直接转移

模拟退火 Simulated Annealing/SA

为了逃离局部最大值，可以对爬山算法取不同的初始值进行（重启）

也可以，引入一个温度参数。随机选择一个邻居并评估，如果更优就选择，但是如果更差，在温度高的时候，也可能会选择移动，温度越高可能性越大；温度越低，表现越接近爬山

```
def simulated_annealing(initial_solution, initial_temp=1000,
                        cooling_rate=0.95, max_iterations=1000):
    """
    模拟退火算法

    Args:
        initial_temp: 初始温度
        cooling_rate: 冷却率 (0 < rate < 1)
    """
    current = initial_solution
    current_value = evaluate(current)
    best = current
    best_value = current_value

    temperature = initial_temp

    for i in range(max_iterations):
        if temperature < 0.01: # 温度足够低时停止
            break

        # 随机选择一个邻居
        neighbor = random.choice(get_neighbors(current))
        neighbor_value = evaluate(neighbor)

        # 计算能量差（假设最小化问题）
```

```
delta = current_value - neighbor_value

if delta > 0:
    # 新解更好，接受
    current = neighbor
    current_value = neighbor_value

    # 更新全局最优
    if neighbor_value < best_value:
        best = neighbor
        best_value = neighbor_value
else:
    # 新解更差，以一定概率接受（Metropolis准则）
    probability = math.exp(delta / temperature)
    if random.random() < probability:
        current = neighbor
        current_value = neighbor_value

# 降低温度
temperature *= cooling_rate

return best, best_value
```

如果温度下降的速度足够慢，这个算法保证optimal，但是实际上这是不可能的

遗传算法 Genetic Algorithms/GA

略